```
%%writefile matrix_mul.cu
#include <iostream>
#include <cuda.h>
#include <cstdlib>
#include <ctime>

using namespace std;

#define TILE_SIZE 16 // 16x16 threads per block

// ------------------------------------------------------------
// CUDA Kernel for Matrix Multiplication
// ------------------------------------------------------------
__global__ void matrixMultiply(int *A, int *B, int *C, int N) {
    // Calculate global row and column
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if (row < N && col < N) {
        int sum = 0;
        for (int k = 0; k < N; k++) {
            sum += A[row * N + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
    }
}

// ------------------------------------------------------------
// Helper Function to Print Matrices
// ------------------------------------------------------------
void printMatrix(int *mat, int N, const char* name) {
    cout << "\n--- Matrix " << name << " ---\n";
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            cout << mat[i * N + j] << "\t";
        }
        cout << "\n";
    }
}

// ------------------------------------------------------------
// Main Function & Benchmarking
// ------------------------------------------------------------
int main() {
    int N;
    cout << "Enter the size of the square matrix (N x N): ";
    cin >> N;

    size_t size = N * N * sizeof(int);
```

```
// Allocate memory on Host
int *h_A = (int *)malloc(size);
int *h_B = (int *)malloc(size);
int *h_C = (int *)malloc(size);

// Smart Input Logic
if (N <= 5) {
    char choice;
    cout << "Do you want to enter elements manually? (y/n): ";
    cin >> choice;

    if (choice == 'y' || choice == 'Y') {
        cout << "Enter " << N * N << " elements for Matrix A (r
        for (int i = 0; i < N * N; i++) cin >> h_A[i];

        cout << "Enter " << N * N << " elements for Matrix B (r
        for (int i = 0; i < N * N; i++) cin >> h_B[i];
    } else {
        srand(time(0));
        for (int i = 0; i < N * N; i++) {
            h_A[i] = rand() % 10;
            h_B[i] = rand() % 10;
        }
    }
} else {
    // Auto-generate for massive benchmark sizes to save time
    srand(time(0));
    for (int i = 0; i < N * N; i++) {
        h_A[i] = rand() % 10;
        h_B[i] = rand() % 10;
    }
    cout << "[Auto-generating " << N << "x" << N << " matrices f
}

// Print input matrices if they are small enough
if (N <= 5) {
    printMatrix(h_A, N, "A");
    printMatrix(h_B, N, "B");
}

// Allocate memory on Device
int *d_A, *d_B, *d_C;
cudaMalloc((void **)&d_A, size);
cudaMalloc((void **)&d_B, size);
cudaMalloc((void **)&d_C, size);

// Copy from Host to Device
cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

// Define Grid and Block dimensions (2D)
```

```
    dim3 threadsPerBlock(TILE_SIZE, TILE_SIZE);
    dim3 blocksPerGrid((N + TILE_SIZE - 1) / TILE_SIZE, (N + TILE_S

    // Set up CUDA Events
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    // Launch Kernel and record time
    cudaEventRecord(start);
    matrixMultiply<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_
    cudaEventRecord(stop);

    cudaEventSynchronize(stop);

    float milliseconds = 0;
    cudaEventElapsedTime(&milliseconds, start, stop);

    // Copy result back to Host
    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

    // Verification Output
    if (N <= 5) {
        printMatrix(h_C, N, "C (Result: A * B)");
    } else {
        cout << "\n[Success] " << N << "x" << N << " Matrix Multipl
    }

    cout << "\n[TIME] GPU Execution Time: " << milliseconds << " ms

    // Free memory
    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    free(h_A); free(h_B); free(h_C);

    return 0;
}
```

```
Overwriting matrix_mul.cu
```

```
!nvcc matrix_mul.cu -o matrix_mul
!./matrix_mul
```

```
nvcc warning : Support for offline compilation for architectures prior to '<compu
Enter the size of the square matrix (N x N): 2
Do you want to enter elements manually? (y/n): y
Enter 4 elements for Matrix A (row by row):
1 2
3 4
Enter 4 elements for Matrix B (row by row):
5 6
7 8
```

```
--- Matrix A ---
1       2
3       4

--- Matrix B ---
5       6
7       8

--- Matrix C (Result: A * B) ---
19      22
43      50

[TIME] GPU Execution Time: 0.202752 ms
```

Start coding or generate with AI.